# What are Hash Functions and How to choose a good Hash Function?

Last Updated : 08 Feb, 2025

## What is a Hash Function?

A hash function is a function that converts a given large number (such as a phone number) into a smaller, practical integer value. This mapped integer value is used as an index in a hash table. In simple terms, a hash function maps a large number or string to a small integer that can be used as the index in the hash table.

## What is Meant by a Good Hash Function?

A good hash function should have the following properties:

- **Efficiently Computable**: The function should be fast to compute.
- **Uniform Distribution of Keys**: The hash function should distribute the keys evenly across the hash table (each table position should be equally likely for each key).

For example, for phone numbers, a bad hash function would be to take the first three digits, while a better hash function would use the last three digits. However, this may not always be the best approach. There may be other more efficient ways to design a hash function.

## Rules for Choosing a Good Hash Function:

1. **Simplicity**: The hash function should be simple to compute.
2. **Minimise Collisions**: The number of collisions should be minimised when placing a record in the hash table. Ideally, no collision should occur, which would make it a perfect hash function.
3. **Uniform Distribution**: The hash function should produce keys that get distributed uniformly over the hash table.

4. **Consider All Bits of the Key**: The hash function should depend on every bit of the key. A function that extracts only a portion of the key is not suitable.

In practice, heuristic techniques can often be employed to create a hash function that performs well. Information about the distribution of the keys may be helpful in designing the function. A good hash function should depend on every single bit of the key to ensure that small changes (even a single bit difference) lead to different hash values.

If two keys differ in just one bit, or if they are permutations of each other (such as 139 and 319), they should hash to different values.

### Heuristic Methods for Hashing

**1. Hashing by Division**: In this method, we map a key to one of the slots of a hash table by taking the remainder when dividing the key by the table size. The hash function can be represented as:

> *h(key) = key % table_size*

Since division is computationally fast, hashing by division is quite efficient. However, some values of `table_size` should be avoided. For example, if `table_size` is a power of a number (e.g., `2^k`), then the hash function may not distribute the keys evenly.

**For example:** Suppose the key `37599` is mapped using a table size of `17`:

> *h(37599) = 37599 % 17 = 12*
>
> *However, for key 573:*
> *h(573) = 573 % 17 = 12*

This leads to a **collision** because both keys are mapped to the same hash value. To avoid this, the table size should ideally be a prime number, and it should not be close to an exact power of 2.

**2. The Multiplication Method**: In the multiplication method, we multiply the key `k` by a constant real number `c` in the range `0 < c < 1`, and then extract the fractional part of the result. We then multiply this value by the table size `m` and take the floor of the result. The hash function is given by:

*h(k) = floor(m * (k * c mod 1))*

*Alternatively, this can also be written as:*

*h(k) = floor(m * frac(k * c)), where `floor(x)` is the integer part of `x`, and `frac(x)` is the fractional part of `x` (i.e., `frac(x) = x - floor(x)`).*

An advantage of the multiplication method is that the value of `m` is not critical. It's typically chosen as a power of 2 (e.g., `m = 2^p`) for simplicity, as this is easy to implement on most computers.

The constant `c` is often chosen to be the fraction `(sqrt(5) - 1) / 2 = 0.618033988...` because this value works reasonably well.

**Example:** Suppose `k = 123456`, `m = 2^14 = 16384`, and `w = 32` (where `w` is the word size of the machine). Then we calculate:

*key * s = 327706022297664 = (76300 * 2^32) + 17612864*
*r1 = 76300, r0 = 17612864*
*The 14 most significant bits of r0 yield the hash value:*
*h(key) = 67*



Article Tags : DSA Functions

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem